1) Half adder and Full subtractor-lab prog

2) Half subtractor and Full adder-lab prog

3) 8x1 Mux using behavioral modelling

```
module mux(x,s,y);
input [7:0]x;
input [2:0]s;
output y;
reg y;
always@(x or s)
begin
if(s==0)
y=x[0];
else if(s==1)
y=x[1];
else if(s==2)
y=x[2];
else if(s==3)
y=x[3];
else if(s==4)
y=x[4];
else if(s==5)
y=x[5];
else if(s==6)
y=x[6];
else if(s==7)
y=x[7];
end
endmodule


module testbench();
reg [7:0]x;
reg [2:0]s;
wire y;
mux m1(x,s,y);
initial
begin
x=8'b00000001;s=3'd0;
#10 x=8'b00000010;s=3'd1;
#10 $finish;
end
endmodule
```

4) 8x1 Mux using 4x1 Mux

```
module mux4(i,s,y);
input [3:0]i;
input [1:0]s;
output y;
reg y;
always @(i or s)
case(s)
                2'b00: y=i[0];
                2'b01: y=i[1];
                2'b10: y=i[2];
                2'b11: y=i[3];
endcase
endmodule

module mux8(x,sel,y0);
input [7:0]x;
input [2:0]sel;
output y0;
wire y1,y2;
mux4 m1(x[3:0],sel[1:0],y1);
mux4 m2(x[7:4],sel[1:0],y2);
assign y0=sel[2]?y2:y1;
endmodule

module testbench();
reg [7:0]x;
reg[2:0]sel;
wire y0;
mux8 m3(x,sel,y0);
initial
begin
x=8'b11111111;sel=3'd0;
#10 x=8'b00000000;sel=3'd0;
#10 $finish;
end
endmodule
```

5) 3x8 Decoder-lab prog

6) 4x16 Decoder

```
module dec4x16(x,y);
input [3:0]x;
output [15:0]y;
assign y[0]= ~x[0] & ~x[1] & ~x[2] & ~x[3];
assign y[1]= ~x[0] & ~x[1] & ~x[2] &  x[3];
assign y[2]= ~x[0] & ~x[1] &  x[2] & ~x[3];
assign y[3]= ~x[0] & ~x[1] &  x[2] &  x[3];
assign y[4]= ~x[0] &  x[1] & ~x[2] & ~x[3];
assign y[5]= ~x[0] &  x[1] & ~x[2] &  x[3];
```

```
assign y[6]= ~x[0] &  x[1] &  x[2] & ~x[3];
assign y[7]= ~x[0] &  x[1] &  x[2] &  x[3];
assign y[8]=  x[0] & ~x[1] & ~x[2] & ~x[3];
assign y[9]=  x[0] & ~x[1] & ~x[2] &  x[3];
assign y[10]= x[0] & ~x[1] &  x[2] & ~x[3];
assign y[11]= x[0] & ~x[1] &  x[2] &  x[3];
assign y[12]= x[0] &  x[1] & ~x[2] & ~x[3];
assign y[13]= x[0] &  x[1] & ~x[2] &  x[3];
assign y[14]= x[0] &  x[1] &  x[2] & ~x[3];
assign y[15]= x[0] &  x[1] &  x[2] &  x[3];
endmodule

module testbench();
reg [3:0]x;
wire [15:0]y;
dec4x16 test(x,y);
initial
begin
x=4'b0000;
#10 x=4'b0001;
#10 x=4'b0010;
..
..
..
#10 x=4'b1111;
#10 $finish;
end
endmodule
```

7) 1:8 Demux (1:4 demux-lab prog)

```
module demux8(x,s,y);
input x;
input [2:0]s;
output [7:0]y;
reg [7:0]y;
always @ (x or s)
begin
y=8'd0;
case(s)
                3'b000:y[0]=x;
                3'b001:y[1]=x;
                3'b010:y[2]=x;
                3'b011:y[3]=x;
                3'b100:y[4]=x;
                3'b101:y[5]=x;
                3'b110:y[6]=x;
                3'b111:y[7]=x;
endcase
end
```

```
endmodule

module testbench();
reg x;
reg [2:0]s;
wire [7:0]y;
demux8 d1(x,s,y);
initial
begin
x=1;s=3'd0;
#10 x=1;s=3'd5;
#10 $finish;
end
endmodule
```

8)BCD to Seven segment common cathode-lab prog

here the cathodes of the seven led's are connected to ground and hence u need a high pulse at the anode to excite the required led.

eg., to represent 0---------1111110(abcdefg)

9)BCD to Seven segment common anode

lll[y] here the anodes of the seven led's are connected to supply and hence u need a gnd pulse at the cathode to excite the required led.

eg., to represent 0---------0000001(abcdefg)

10) Weighted code to Cyclic code(Binary to gray-lab prog)

11) Cyclic to Weighted(gray to binary-lab prog)

12) Weighted to Self Complimenting(Binary to excess3-lab prog)

13) Self Complimenting to Weighted(exs3 to binary-lab prog)

14)Full adder using half adder-lab prog

15) 4bit magnitude comparator using 2bit mag comp

```
module mag2(a1,b1,g,e,l);
input [1:0]a1,b1;
output g,e,l;
assign g=(a1>b1),
       e=(a1==b1),
       l=(a1<b1);
endmodule

module mag4(a,b,g,e,l);
input [3:0]a,b;
output g,e,l;
```

```
wire g1,e1,l1,g2,e2,l2;
mag2 m1(a[3:2],b[3:2],g1,e1,l1);
mag2 m2(a[1:0],b[1:0],g2,e2,l2);
assign g=(g1 | (g2&e1)),
      l=(l1 | (l2&e1)),
      e=(e1 & e2);
endmodule

module testbench();
reg [3:0]a,b;
wire g,e,l;
mag4 m(a,b,g,e,l);
initial
begin
a=4'b0010;b=4'b1001;
#10 a=4'b1100;b=4'b1100;
#10 a=4'b1100;b=4'b1011;
#10 $finish;
end
endmodule
```

16) 4bit Ripple carry adder

```
module fa(a1,b1,c1,s,c0);
input a1,b1,c1;
output s,c0;
assign {c0,s}=a1 + b1 + c1;
endmodule

module rca(a,b,c,s,c0);
input [3:0]a,b;
input c;
output [3:0]s;
output c0;
wire t1,t2,t3;
fa f1(a[0],b[0],c,s[0],t1);
fa f2(a[1],b[1],t1,s[1],t2);
fa f3(a[2],b[2],t2,s[2],t3);
fa f4(a[3],b[3],t3,s[3],c0);
endmodule

module testbench();
reg [3:0]a,b;
reg c;
wire [3:0]s;
wire c0;
rca r(a,b,c,s,c0);
initial
begin
a=4'b1000;b=4'b0101;c=0;
```

```
#10 a=4'b0101;b=4'b1000;c=1;
#10 a=4'b1001;b=4'b1000;c=0;
#10 $finish;
end
endmodule
```

17) 4:3 Encoder-lab prog

18) Asyncronous 4bit counter using JK flipflop

In asynchronous the clock pulse is not common to all flipflops thus is given only to one flipflop and its output triggers the next flipflop. Here Q0 triggers the second JKFF.

```
module jkff(j,k,clk,clr,q);
input j,k,clr,clk;
output q;
reg q;
always @ (posedge clk or negedge clr)
begin
if(clr==0)
q=0;
else
begin
assign q=(j & ~q) | (~k & q);
end
end
endmodule

module asyncntr(clk,clr,Q);
input clk,clr;
output [3:0]Q;
reg Q=4'b0000;
wire j1,k1;
assign j1=1,
       k1=1;
jkff ff1(j1,k1,clk,clr,Q[0]);
jkff ff2(j1,k1,~Q[0],clr,Q[1]);
jkff ff3(j1,k1,~Q[1],clr,Q[2]);
jkff ff4(j1,k1,~Q[2],clr,Q[3]);
endmodule

module testbench();
reg clk,clr;
wire [3:0]Q;
asyncntr test(clk,clr,Q);
initial
begin
clk=1;
forever #5 clk=~clk;
end
```

```
initial
begin
clr=0;
#10 clr=1;
#10 clr=1;
#100 $finish;
end
endmodule
```

19) 4bit Syncronous counter using T flipflop

Design:

Present state q3q2q1q0	Next state Q3Q2Q1Q0	Exitation table T3T2T1T0
0000	0001	0001
0001	0010	0011
0010	0011	0001
0011	0100	0111
0100	0101	0001
..		
..		
1111	0000	1111

using k map

➔T3=q2.q1.q0

➔T2=q1.q0

➔T1=q0

➔T0=1

```
module tff(t,clk,clr,q);
input t,clr,clk;
output q;
reg q;
always @ (posedge clk or posedge clr)
begin
if(clr==1)
q=0;
else
begin
if (t==0)
assign q=q;
else
assign q=~q;
end
end
endmodule

module syncntr(clk,clr,q);
input clk,clr;
output [3:0]q;
wire t0,t1,t2,t3;
assign t0=1,
```

```
      t1=q[0],
                  t2=(q[1] & q[0]),
                  t3=(q[2] & q[1] &q[0]);
tff ff1(t0,clk,clr,q[0]);
tff ff2(t1,clk,clr,q[1]);
tff ff3(t2,clk,clr,q[2]);
tff ff4(t3,clk,clr,q[3]);
endmodule

module testbench();
reg clk,clr;
wire [3:0]q;
syncntr test(clk,clr,q);
initial
begin
clk=1;
forever #5 clk=~clk;
end
initial
begin
clr=1;
#10 clr=0;
#100 $finish;
end
endmodule
```

20) Converting JK into SR(SR using JK)

```
module jkff(j,k,clk,clr,q);
input j,k,clr,clk;
output q;
reg q=0;
always @ (posedge clk or clr)
begin
if(clr==1)
q=0;
else
begin
if (j==0 && k==0)
        assign q=q;
else if(j==0 && k==1)
        assign q=0;
else if(j==1 && k==0)
        assign q=1;
else if(j==1 && k==1)
        assign q=~q;
end
end
endmodule
```

```
module sr(s,r,clk,clr,q1);
input s,r,clk,clr;
output q1;
reg q1;
wire j1,k1;
assign j1=(s & ~r);
assign k1=(r & ~s);
jkff test1(j1,k1,clk,clr,q1);
endmodule

module testbench();
reg s,r,clk1,clr1;
wire q1;
sr test(s,r,clk1,clr1,q1);
initial
begin
clk1=1;
forever #5 clk1=~clk1;
end
initial
begin
clr1=1; s=0; r=0;
#10 clr1=0;s=0;r=0;
#10 clr1=0;s=0;r=1;
#10 clr1=0;s=1;r=0;
#10 clr1=0;s=1;r=1;
#10 $finish;
end
endmodule
```

20) Converting JK into T(T using JK)

```
module jkff(j,k,clk,clr,q);
input j,k,clr,clk;
output q;
reg q=0;
always @ (posedge clk or clr)
begin
if(clr==1)
q=0;
else
begin
if (j==0 && k==0)
        assign q=q;
else if(j==0 && k==1)
        assign q=0;
else if(j==1 && k==0)
        assign q=1;
else if(j==1 && k==1)
        assign q=~q;
end
```

```
end
endmodule


module tusingjk(t,clk,clr,q);
input t,clk,clr;
output q;
reg q;
wire j1,k1;
assign j1=t;
assign k1=t;
jkff test1(j1,k1,clk,clr,q);
endmodule

module testbench();
reg t,clk,clr;
wire q;
tusingjk test(t,clk,clr,q);
initial
begin
clk=1;
forever #5 clk=~clk;
end
initial
begin
clr=1; t=0;
#10 clr=0;t=0;
#10 clr=0;t=1;
#10 $finish;
end
endmodule
```

20) Converting JK into D(D using JK)

```
module jkff(j,k,clk,clr,q);
input j,k,clr,clk;
output q;
reg q=0;
always @ (posedge clk or clr)
begin
if(clr==1)
q=0;
else
begin
if (j==0 && k==0)
        assign q=q;
else if(j==0 && k==1)
        assign q=0;
else if(j==1 && k==0)
        assign q=1;
```

```
else if(j==1 && k==1)
        assign q=~q;
end
end
endmodule

module dusingjk(d,clk,clr,q);
input d,clk,clr;
output q;
reg q;
wire j1,k1;
assign j1=d;
assign k1=~d;
jkff test1(j1,k1,clk,clr,q);
endmodule

module testbench();
reg d,clk,clr;
wire q;
dusingjk test(d,clk,clr,q);
initial
begin
clk=1;
forever #5 clk=~clk;
end
initial
begin
clr=1; d=0;
#10 clr=0;d=0;
#10 clr=0;d=1;
#10 $finish;
end
endmodule
```

21)PIPO Shift register-lab prog

22)SISO Shift register-lab prog

23)SIPO Shift register-lab prog

24)Full subtractor using half sub-lab prog

25)FPGA Real time clock-lab prog

26) FPGA Parallel adder-lab prog

27)FPGA Multiplier-lab prog

28,29,30,31) Parity generator and checker 4bit-labprogs

32,33) Jhonson counter and Ring counter-lab progs

34) 3bit Upcounter

```
module counter(clk,c0);

input clk;

output [2:0]c0;

reg c0=0;

always @(posedge clk)

c0=c0+1;

endmodule


module testbench();

reg clk;

wire [2:0]c0;

counter c1(clk,c0);

initial

begin

clk=0;

forever #100 clk=~clk;

end

endmodule
```

3bit down counter

```
module counter(clk,c0);

input clk;

output [2:0]c0;

reg c0=0;

always @(posedge clk)

c0=c0-1;
```

endmodule

module testbench();

reg clk;

wire [2:0]c0;

counter c1(clk,c0);

initial

begin

clk=0;

forever #100 clk=~clk;

end

endmodule

35,36) D and JK flipflop-lab progs

37) Mod7 counter

```
module mod7ctr(clk,cnt);
input clk;
output cnt;
reg [3:0]cnt;
always @(posedge clk)
begin
if (cnt !=4'd7)
    assign cnt=cnt+1;
       else
       assign cnt=0;
end
endmodule

module testbench();
wire [3:0]cnt;
reg clk;
assign cnt=4'd0;
mod7ctr test (clk,cnt);
initial
begin
clk=0;
forever #5 clk=~clk;
end
initial
begin
#100 $finish;
```

```
end
endmodule
```

38,39,40) Mod10counter & bcd to ex3 & ex3 to bcd-lab progs

43) T flipflop-lab progs

44) 3bit upcounter-see 34)

45) 8x1 Mux

same as lab prog using clock

please upload the progs 41 and 42

ALL THE BEST.